PaddleHubs - Phase 1 Completion Document

Founder: Sai Sidharth Vinothkannan
Status: Completed and Production-Ready
Website: paddlehubs.com

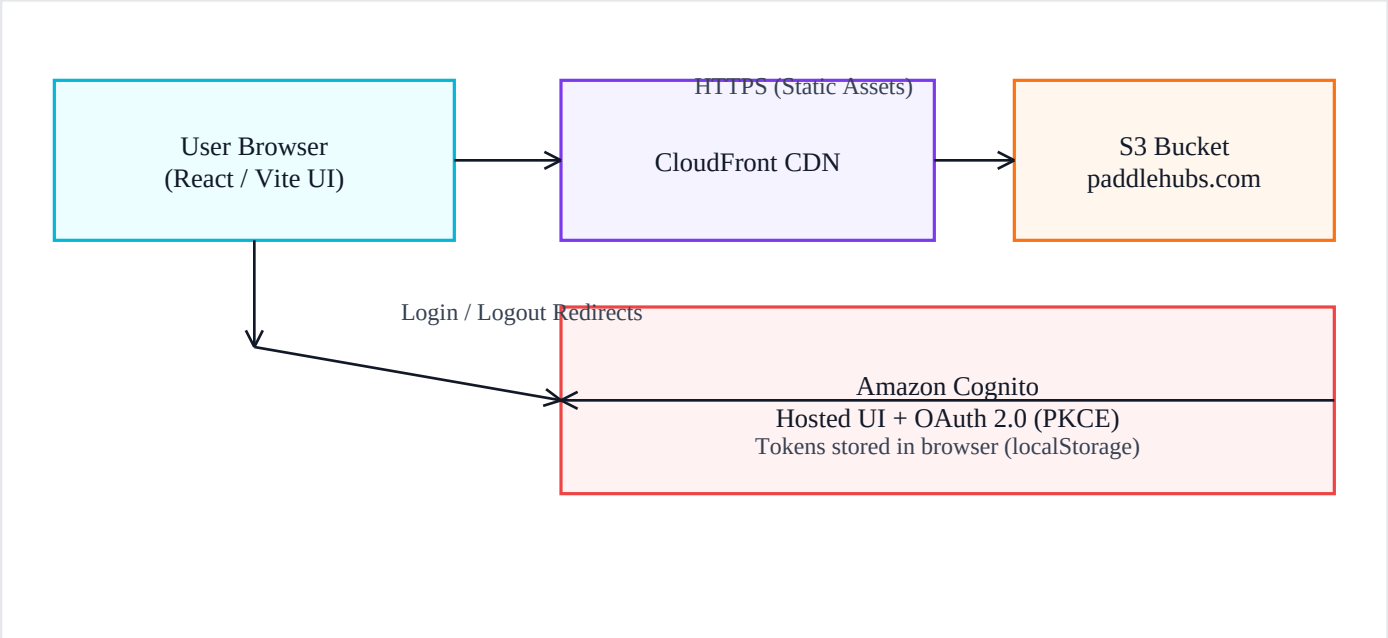
Version: 1.3 (includes architecture diagram, code snippets, file inventory, and step-by-step commands)

1. Executive Summary

Phase 1 delivers a secure, cloud-hosted pickleball club portal with a rich, responsive UI and user authentication. Users can log in via Amazon Cognito, then access protected sections for Court Booking and Match Details. Phase 1 validates the end-to-end user flow (login, navigation, and feature workflows) and establishes a stable production deployment on AWS.

2. Architecture Diagram (Phase 1)

High-level AWS architecture used in Phase 1:



Hosting: S3 + CloudFront (static SPA)
Authentication: Amazon Cognito Hosted UI with OAuth 2.0 Authorization Code Grant + PKCE
Routing: React Router with SPA fallback configured in CloudFront/S3

3. Feature Scope (Phase 1)

Authentication: Login and logout via Cognito Hosted UI; protected routes enforced in the app. **Court Booking (UI):** Form and page flow available for authenticated users. **Match Details (UI):** Singles and doubles match entry; user inputs player names; match history UI. **Responsive Layout:** Sidebar navigation (desktop) and drawer navigation (mobile).

4. Repository Structure and Files Used

```
src/
  App.jsx
  main.jsx
  components/
    Layout.jsx
    RequireAuth.jsx
  lib/
    auth.js
  pages/
    Dashboard.jsx
    CourtBooking.jsx
    MatchDetails.jsx
    AuthCallback.jsx
```

File	Purpose (Phase 1)
src/main.jsx	Creates the React root and wraps the app with BrowserRouter (single router).
src/App.jsx	Defines public routes and protected routes for Court Booking and Match Details.
src/components/Layout.jsx	Top bar + side menu; Login/Logout buttons; shows menu items only when logged in.
src/components/RequireAuth.jsx	Guards protected pages; redirects unauthenticated users to '/'.
src/lib/auth.js	Cognito login URL (PKCE), logout URL, code-to-token exchange, token storage helpers.
src/pages/AuthCallback.jsx	Handles /auth/callback; exchanges code for tokens; redirects to a protected page.
src/pages/CourtBooking.jsx	Court booking UI (Phase 1: client-side only).
src/pages/MatchDetails.jsx	Singles/doubles match UI and match history (Phase 1: client-side only).

4.1 Key Code Snippets

Cognito login URL builder (PKCE flow) — `src/lib/auth.js`:

```
export function loginUrl() {
  const domain = import.meta.env.VITE_COGNITO_DOMAIN;
  const clientId = import.meta.env.VITE_COGNITO_CLIENT_ID;
  const redirectUri = import.meta.env.VITE_COGNITO_REDIRECT_URI;
  const params = new URLSearchParams({
    client_id: clientId,
    response_type: "code",
    scope: "openid email profile",
    redirect_uri: redirectUri,
  });
```

```
  return `${domain}/login?${params.toString()}`;
}
```

Protected route guard — `src/components/RequireAuth.jsx`:

```
import { Navigate } from "react-router-dom";
import { isLoggedIn } from "../lib/auth";

export default function RequireAuth({ children }) {
  if (!isLoggedIn()) return <Navigate to="/" replace />;
  return children;
}
```

5. Cognito Configuration (Phase 1)

The following Cognito settings are required for PaddleHubs Phase 1 to function correctly:

Hosted UI Domain: https://auth.paddlehubs.com **OAuth Flow:** Authorization code grant (PKCE) enabled **Scopes:** openid, email, profile **Callback URL:** https://paddlehubs.com/auth/callback **Sign-out URL:** https://paddlehubs.com

Environment variables used by the frontend:

```
VITE_COGNITO_DOMAIN=https://auth.paddlehubs.com
VITE_COGNITO_CLIENT_ID=YOUR_CLIENT_ID
VITE_COGNITO_REDIRECT_URI=https://paddlehubs.com/auth/callback
VITE_COGNITO_LOGOUT_URI=https://paddlehubs.com
```

6. Step-by-Step Commands (EC2 + S3 + CloudFront)

Run these commands on the EC2 instance in the project directory:

```
# install dependencies
npm install

# build production bundle
npm run build

# sync to S3 (static hosting bucket)
aws s3 sync dist/ s3://paddlehubs.com --delete

# (if using CloudFront) invalidate cache so new JS/CSS loads immediately
aws cloudfront create-invalidation --distribution-id YOUR_DISTRIBUTION_ID --paths "/*"
```

Update the GitHub repository (from the same project folder):

```
# push latest code to GitHub
git status
git add -A
git commit -m "Phase 1: Cognito login + protected routes + UI"
git push origin main
```

SPA routing requirement: configure CloudFront to rewrite unknown paths to `/index.html`. In CloudFront, add custom error responses for 403 and 404 → Response page path: /index.html → HTTP response code: 200.

7. Phase 1 Completion Checklist

Website hosted at paddlehubs.com (S3 + CloudFront) Cognito login and logout working end-to-end Protected routes working: /court-booking and /match-details Clean UI and responsive navigation (mobile + desktop) Stable routing on refresh (SPA fallback configured)

8. Phase 2 Preview (Data Persistence + User Linking)

Phase 2 will add persistent storage and link each match/booking record to the logged-in user. Recommended approach: store bookings and matches in DynamoDB, expose CRUD operations through API Gateway + Lambda, and attach the user's Cognito subject (sub) and email to every record.